

Specific Methods

Normal Distribution

That is, a normal random variable can be simulated using the sum of n random variables $\{u_i\}_{i=1}^n \in \mathcal{U}[0, 1]$.

Since $\mathbb{E}[u_i] = 0,5$ and $\text{Var}(u_i) = \frac{1}{12}, \forall i = 1, \dots, n$, we have

$$Z = \frac{\sum_{i=1}^n u_i - \frac{n}{2}}{\sqrt{\frac{n}{12}}} = \frac{X - m_X}{\sigma_X},$$

From which we obtain

$$X = \left(\sum_{i=1}^n u_i - \frac{n}{2} \right) \sigma_X \sqrt{\frac{12}{n}} + m_X, \forall i = 1, n.$$

Remark: Preferably, n should be large (generally $n \geq 50$); however, for satisfactory results, we take $n = 12$.

Specific Methods

The Box et Muller approach for Normal Distribution

This is a popular method for generating Gaussian variables. Its advantage lies in its simplicity, but the quality of the generated values depends on that of the uniform variables. It allows the generation of an independent pair of random variables Z_1, Z_2 each following a standard normal distribution. Then, we can obtain two realizations of the variable X distributed according to $\mathcal{N}(m_X, \sigma_X)$ using $X_1 = \sigma_X Z_1 + m_X$ and $X_2 = \sigma_X Z_2 + m_X$.

Specific Methods

The Box et Muller approach for Normal Distribution

Algorithme

- 1 Generate u_1 and u_2 according to independent uniform distributions on $(0, 1)$.
- 2 Calculate z_1 and z_2 using the following formulas:

$$Z_1 = \sqrt{-2 \ln(u_1)} \cos(2\pi u_2) \text{ and } Z_2 = \sqrt{-2 \ln(u_1)} \sin(2\pi u_2)$$
- 3 Return $X_1 = \sigma_X Z_1 + m_X$ and $X_2 = \sigma_X Z_2 + m_X$.

Generating Random Vectors

Suppose we need to generate a random vector

$X = (X_1, X_2, \dots, X_n)$ from n given probability density distributions $f(x)$ and distribution function $F(x)$. When the components $X_1, X_2, \dots, X_n$ are **independent**, the situation is straightforward: we simply apply the inverse transformation method or another generation method of our choice to each component individually.

Generating Random Vectors

Conditioning algorithm

Given dependent random variables $X_1, \dots, X_n$ their joint density can be expressed using the conditional probability formula as:

$$f(x_1, x_2, \dots, x_n) = f_1(x_1) \times f_2(x_2/x_1) \times \dots \times f_n(x_n/x_1, \dots, x_{n-1})$$

where $f_1(x_1)$ is the marginal density of X_1 and $f_i(x_i/x_1, \dots, x_{i-1})$ are the conditional densities of X_i given $X_1 = x_1, \dots, X_{i-1} = x_{i-1}$. The algorithm to generate a random vector $(X_1, \dots, X_n)$ is as follows:

Generating Random Vectors

Conditioning algorithm

Algorithm

- 1 Initialization: Set $i \leftarrow 1$.
- 2 Draw $u_1 \in \mathcal{U}[0, 1]$;
- 3 While $i \leq n$:
 - a. If $i = 1$, draw X_1 using the marginal density $f_1(x_1)$, (applying a suitable transformation).
 - b. If $i > 1$, generate X_i with density $f_i(x_i/x_1, \dots, x_{i-1})$, applying an appropriate simulation method.
 - c. Increment $i \leftarrow i + 1$.
- 4 If $i < n$, return to 1;
- 5 When $i > n$, return the vector $(X_1, \dots, X_n)$.

Multivariate Gaussian Vector Generation

If $\mathbf{X} = (X_1, X_2, \dots, X_n) \sim \mathcal{N}(\mathbf{m}, \mathbf{C})$;

$$f(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{n}{2}} |\mathbf{C}|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mathbf{m})^T \mathbf{C}^{-1} (\mathbf{x} - \mathbf{m}) \right) \quad (14)$$

$$\mathbf{C} \text{ is } \begin{cases} \text{Symmetric} \\ \text{Positive Defined} \end{cases} \Rightarrow \mathbf{C} = \mathbf{A}\mathbf{A}^T \quad (15)$$

The idea behind generating a multivariate (or simply multi-normal) normal random vector is to write it as $\mathbf{X} = \mathbf{m} + \mathbf{A}\mathbf{Z}$, where $\mathbf{A}$ is a matrix such that $\mathbf{A}\mathbf{A}^T = \mathbf{C}$, and $\mathbf{Z}$ is a vector of random variables i.i.d. $\mathcal{N}(0, 1)$.

Multivariate Gaussian Vector Generation

Cholesky Decomposition

Algorithme

Input: Mean vector m and covariance matrix C .

Output: Random vector $X \sim \mathcal{N}(m, C)$.

Given: the Cholesky decomposition AA^T of C .

- 1 Generate $Z = (Z_1, Z_2, \dots, Z_n)$ with distribution $\mathcal{N}(0, I)$;
- 2 Assume $X = m + AZ$.

Discrete Time Markov Chain Simulation

Let a Markov chain $\{X_n, n \geq 0\}$ with finite states space $S = \{1, 2, \dots, n\}$ (or countable) of transition matrix $P = (p_{ij})_{n \times n}$, where

$$p_{ij} = P(X_n = j / X_{n-1} = i), i = 1, \dots, n \text{ and } \sum_{i=1}^n p_{ij} = 1.$$

Discrete Time Markov Chain Simulation

Algorithm

① Divide the interval $[0, 1]$ in two ways:

a) To generate the distribution of the initial state:

$[0, 1] = \sum_{i=1}^n |\Delta_i^0|$; the measure of the interval

$|\Delta_i^0| = \pi_i^0, i = 1, \dots, n; \Delta_i^0 = \sum_{k=1}^i \Delta_k^0$ (this is the cumulative distribution function).

b) To generate the current state j , given the previous state i :

$[0, 1] = \sum_{i=1}^n |\Delta_{ij}|$; the measure of the interval

$|\Delta_{ij}| = p_{ij}, i = 1, \dots, n; \Delta_{ij} = \sum_{k=1}^j \Delta_{ik}.$

Discrete Time Markov Chain Simulation

Algorithme

2. The successive realizations of the Markov chain $\{X_n, n \geq 0\}$ are determined using the recursive formulas:

$$X_0 = i, \text{ if } u_0 \in \Delta_i;$$

$$X_n = j, \text{ if } u_n \in \Delta_{X_{n-1},j}; n \geq 1.$$

Continuous Time Markov Chain Simulation

Let $\{X(t), t \geq 0\}$ be a homogeneous Markov process with a discrete or countable states space S , and the initial distribution is $\pi_i^0 = P(X_0 = i)$ with $\sum_i \pi_i^0 = 1$.

If the derivatives exist, the transition rates are defined as

$$\lambda_{ij} = \lim_{dt \rightarrow 0} \frac{p_{ij}(dt)}{dt}, i \neq j, \lambda_i = \sum_{j \neq i} \lambda_{ij},$$

with condition

$$p_{ij}(0) = \lim_{dt \rightarrow 0} p_{ij}(dt) = \begin{cases} 1, & \text{if } j = i \\ 0, & \text{if } j \neq i \end{cases}.$$

Continuous Time Markov Chain Simulation

The trajectory of such a process is as follows: Let i_0 the initial state determined by the distribution π_i^0 ; the duration of stay in this state is a random variable τ_0 with an exponential distribution $\exp(\lambda_{i_0})$:

$$\mathbb{P}(\tau_0 < x) = 1 - e^{-\lambda_{i_0} x}, x \geq 0.$$

The probability that the next state i_1 is

$$\mathbb{P}(X(\tau_0) = i_1 / X(0) = i_0) = \frac{\lambda_{i_0, i_1}}{\lambda_{i_0}} = p_{i_0, i_1}.$$

The process remains in this state for a time τ_1 following an exponential distribution $\exp(\lambda_{i_1})$:

$$\mathbb{P}(\tau_1 < x) = 1 - e^{-\lambda_{i_1} x}, x \geq 0.$$

Continuous Time Markov Chain Simulation

Once again,

$$\mathbb{P}(X(\tau_0 + \tau_1) = i_2 / X(\tau_0) = i_1) = \frac{\lambda_{i_1, i_2}}{\lambda_{i_1}} = p_{i_1, i_2}$$

and

$$\mathbb{P}(\tau_2 < x) = 1 - e^{-\lambda_{i_2} x}, x \geq 0, \text{ etc...}$$

In other words, the states taken by the process $\{X(t), t \geq 0\}$ during successive transitions form a Markov chain $\{X_n\}$, defined by $X_n = X(\tau_0 + \tau_1 + \dots + \tau_{n-1})$ with transition matrix $P = (p_{ij})$, where $p_{ij} = \frac{\lambda_{ij}}{\lambda_i}$ and initial distribution $\{\pi_i^0\}$.

The variable τ_i denotes the duration of the chain's stay in state i

$$F_i(x) = \mathbb{P}(\tau_i < x) = 1 - e^{-\lambda_i x}, x \geq 0.$$

Continuous Time Markov Chain Simulation

The simulation of the process

$$\{X(t), t \geq 0\},$$

consists of simulating the embedded Markov chain

$$\{X_n, n \geq 1\}$$

at discrete time, where $X_n = X(t_n)$.

Continuous Time Markov Chain Simulation

Algorithme

- 1 Generate $\{U_n\} \in \mathcal{U}[0, 1]$.
- 2 Generate an initial state i_0 according to distribution $\{\pi_i^0\} : X_0 = i \text{ if } U_1 \in \Delta_i$;
- 3 Generate the stay time at this state: $\tau_0 = -\frac{1}{\lambda_{i_0}} \ln U_1$.
- 4 The following states are thus generated by the even terms of the sequence:
 $X_n = j \text{ if } U_{2k} \in \Delta_{n-1,j}, n \geq 1.$
- 5 Generate successive stay times according to the corresponding exponential laws for all odd terms:
 $\tau_n = -\frac{1}{\lambda_{X_n}} \ln U_{2k+1}, n \geq 1.$

Continuous Time Markov Chain Simulation

The process under consideration is then defined by the relationship

$$X(t) = X_n \text{ for } \tau_0 + \tau_1 + \dots + \tau_{n-1} \leq t < \tau_0 + \tau_1 + \dots + \tau_n.$$

This method is often referred to as "**event-by-event simulation**" or "**discrete-event simulation**".